



Base de Datos 2

TPO

Grupo 05

Cortese, Andrés - 64612

Di Candia, Lucas - 63212

Trajterman, Catalina - 64175

1. Introducción.....	3
2. Decisiones de Bases de Datos.....	3
2.1 MongoDB.....	3
2.1.1 Esquema Lógico.....	3
2.1.2 Esquema Físico.....	4
2.2 Neo4j.....	4
2.2.1 Esquema Lógico.....	4
2.2.2 Esquema Físico.....	4
3. Manual de Usuario y Guía de instalación.....	5
- Crear el entorno virtual.....	5
Windows.....	5
4. Resultados de Consultas y Servicios.....	7
4.1 Consultas analíticas.....	7
Q1 — Pacientes activos junto con sus propietarios.....	7
Q2 — Consultas en estado *Seguimiento.....	7
Q3 — Historial clínico unificado de *Firulais* (paciente 1).....	8
Q4 — Propietarios con más de un paciente.....	8
Q5 — Veterinarios con consultas en los últimos 60 días.....	9
Q6 — Pacientes con vacunas vencidas.....	9
Q7 — Top 5 diagnósticos más frecuentes.....	9
Q8 — Productos con stock crítico.....	10
Q9 — Consultas de control con costo menor a \$5.000.....	10
Q10 — Pacientes atendidos por sucursal (*Palermo*).....	11
Q11 — Ingresos por veterinario en el mes actual.....	11
Q12 — Propietarios sin consultas en el último año.....	11
4.2 Servicios operativos.....	12
Q13 — ABM de propietarios (MongoDB + Neo4j).....	12
Q14 — Alta de consulta médica.....	12
Q15 — Decremento de stock farmacéutico.....	13
4.3 Observaciones sobre la consistencia cross-engine.....	13
5. Conclusiones.....	13

1. Introducción

El presente trabajo integra los conceptos abordados en la materia Bases de Datos II del Instituto Tecnológico de Buenos Aires (ITBA), aplicándolos a un caso de estudio real. El objetivo principal es seleccionar y justificar estratégicamente tecnologías NoSQL en función de requerimientos de negocio específicos. Para ello, se diseñó el sistema de *backoffice* de VetSalud S.A., una red de clínicas veterinarias con alcance nacional. La plataforma centraliza la gestión de pacientes, propietarios, profesionales, historias clínicas (consultas, vacunaciones y cirugías) y el stock farmacéutico. A partir de los datos base provistos en formato CSV, el equipo expandió el volumen de información con registros propios para robustecer las pruebas de rendimiento. Finalmente, la arquitectura de persistencia se resolvió mediante un enfoque políglota, combinando de manera óptima dos motores NoSQL para maximizar la eficiencia operativa del negocio.

2. Decisiones de Bases de Datos

Para el diseño de la arquitectura, se analizaron los patrones de consulta y las características de las entidades de VetSalud S.A.. Se concluyó que un modelo híbrido basado en **MongoDB** (orientado a documentos) y **Neo4j** (orientado a grafos) ofrece la mejor sinergia para resolver tanto el almacenamiento operacional y de catálogos como los complejos historiales clínicos interconectados.

2.1 MongoDB

Se seleccionó MongoDB como la base de datos principal para la persistencia operativa e información estructural del backoffice. Su motor está diseñado para garantizar un alto nivel de consistencia a nivel de documento, lo cual es crítico al manipular stock farmacéutico, datos de contacto de clientes y registros maestros de veterinarios.

La principal ventaja que aporta MongoDB en este escenario es su **esquema flexible**. Como los datos de la clínica veterinaria pueden evolucionar rápidamente (incorporando campos específicos según la especie del animal, datos de proveedores o nuevas categorías de medicamentos), el modelo de documentos evita la necesidad de realizar migraciones de tablas estructuradas costosas o complejas (ALTER TABLE). Esto dota al desarrollo de una enorme agilidad adaptativa.

2.1.1 Esquema Lógico

Desde la perspectiva lógica, se decidió modelar en MongoDB aquellas entidades con estructuras de datos complejas pero patrones de acceso independientes o tabulares:

- **propietarios**: Datos demográficos, DNI y contacto.
- **veterinarios**: Matrícula, especialidad y sucursal de atención.
- **stock_farmacéutico**: Inventario detallado, precios y vencimientos por proveedor.

Tras analizar el tradeoff del patrón de consultas, se evitó embeber registros históricos masivos dentro de los documentos principales para mitigar problemas de crecimiento desmedido de documentos. Las colecciones respetan las relaciones de negocio mediante identificadores únicos (id_propietario, id_vet, id_producto).

Para la gestión de clientes en el servicio ABM, se optó por una **baja lógica** (mediante un flag activo: false) en lugar de una eliminación física. Esto preserva el historial clínico de las mascotas, los reportes financieros históricos y evita la existencia de referencias huérfanas en el sistema. Asimismo, se implementó una lógica de apoyo mediante una vista materializada llamada `vw_consultas_seguimiento`, que expone directamente las consultas médicas en estado abierto para agilizar el trabajo diario sin recalcular pipelines de filtrado complejos en cada petición.

2.1.2 Esquema Físico

Físicamente, todas las colecciones se hospedan en una base de datos centralizada de MongoDB, tipificando adecuadamente las claves primarias como datos indexables. Se configuraron índices secundarios sobre:

- unidades en la colección de stock para resolver de forma instantánea alertas de inventario crítico (< 50 unidades).
- estado en las consultas para alimentar la vista operativa de seguimientos de forma eficiente.

2.2 Neo4j

Para complementar la solución, se incorporó **Neo4j** como motor de bases de datos orientado a grafos. Su inclusión está plenamente justificada por la naturaleza interconectada de los historiales médicos (pacientes, vacunas, veterinarios y consultas). Consultas de alta densidad relacional, como la trazabilidad de dueños con múltiples mascotas o el rastreo geográfico de brotes por sucursal, provocarían múltiples operaciones de JOIN o agregaciones pesadas en motores tradicionales; Neo4j las resuelve nativamente mediante punteros físicos en memoria (*index-free adjacency*).

2.2.1 Esquema Lógico

En Neo4j no se duplica la metadata estática del stock, sino que se modela exclusivamente la red de interacciones clínicas y de propiedad. El grafo se compone de los siguientes elementos:

- **Nodos:** (:Paciente), (:Propietario), (:Veterinario), (:Consulta) y (:Vacuna).
- **Relaciones:**
 - (:Propietario)-[:DUEÑO_DE]->(:Paciente)
 - (:Veterinario)-[:REALIZÓ]->(:Consulta)-[:A_PACIENTE]->(:Paciente)
 - (:Veterinario)-[:APLICÓ]->(:Vacuna)-[:A_PACIENTE]->(:Paciente)

Esta abstracción lógica mapea de forma directa el negocio y optimiza las queries de caminos históricos y agrupaciones por proximidad temporal o geográfica.

2.2.2 Esquema Físico

La instancia física se despliega con restricciones de unicidad e índices sobre las propiedades `id_paciente`, `id_propietario` e `id_vet` para garantizar puntos de entrada inmediatos al grafo.

Las propiedades de control como `fecha`, `fecha_aplicacion` y `proxima_dosis` se alojan en los respectivos nodos de eventos (:Consulta y :Vacuna). Al registrarse una nueva atención, el

sistema realiza una escritura dual controlada por la capa de servicios de la API: impacta los detalles en MongoDB y genera en tiempo real los nodos y aristas en Neo4j, logrando una consistencia eventual sólida entre ambos motores.

3. Manual de Usuario y Guía de instalación

A continuación se detallan los pasos para configurar, inicializar y ejecutar todo el entorno de la aplicación.

1. Requisitos Previos

Asegúrate de tener instalados los siguientes componentes:

- Docker y Docker Desktop
- Python 3.10+
- Un cliente de Git (opcional, para clonar el repositorio).

2. Levantar las Bases de Datos (Docker)

La persistencia de datos utiliza MongoDB 7 y Neo4j 5. Contamos con una configuración automatizada mediante Docker Compose.

Para levantar ambos motores, ejecuta el siguiente comando en la raíz del proyecto:

docker-compose up -d=

Esto descargará e iniciará los contenedores necesarios en segundo plano. Puedes verificar que estén corriendo con:

docker compose ps

3. Configurar el Entorno Virtual de Python

Es buena práctica utilizar un entorno virtual para aislar las dependencias del proyecto.

Se recomienda utilizar un entorno virtual para aislar las dependencias del proyecto.

- Crear el entorno virtual

python -m venv venv

- Activar el entorno virtual

Windows

venv\Scripts\activate

Linux / macOS

source venv/bin/activate

- Instalar dependencias

pip install -r requirements.txt

4. Configurar las Variables de Entorno

El proyecto obtiene la configuración de conexión desde un archivo .env.

Crear el archivo a partir del ejemplo:

Linux / macOS

cp .env.example .env

Windows

copy .env.example .env

Configuración por defecto:

MONGO_URI=mongodb://localhost:27017

NEO4J_URI=bolt://localhost:7687

NEO4J_USER=neo4j

NEO4J_PASSWORD=vetsalud123

5. Cargar y Poblar Datos Iniciales (Seed / ETL)

Se dispone de un script orquestador que realiza la ingesta, limpieza e interconexión de datos desde los archivos CSV hacia MongoDB y Neo4j de forma idempotente.

Con el entorno virtual activo ejecutar:

python -m src.etl.seed

Este proceso poblará ambas bases de datos con los datasets originales y los registros adicionales incorporados al proyecto.

6. Ejecutar el Servidor Web (FastAPI)

Una vez que las bases de datos estén activas y los datos cargados, iniciar el servidor mediante Uvicorn:

uvicorn api:app --reload

El servidor quedará disponible en:

<http://127.0.0.1:8000>

7. Acceso a la Documentación Interactiva

Con el servidor en ejecución, acceder desde un navegador a:

<http://127.0.0.1:8000/docs>

Allí se encuentra disponible la interfaz interactiva de Swagger UI, desde la cual es posible probar todas las consultas y servicios del sistema en tiempo real.

4. Resultados de Consultas y Servicios

A partir del dataset base provisto por la cátedra y de los registros extra que generó el equipo, se ejecutaron las quince consultas y servicios diseñados para el backoffice. Los resultados se obtuvieron sobre un entorno controlado (MongoDB 7 y Neo4j 5 levantados con docker compose up -d y poblados con python -m src.etl.seed). Para facilitar la lectura, los reportes se agrupan en dos bloques: las consultas analíticas de solo lectura (Q1 a Q12) y los servicios operativos que escriben en ambas bases (Q13, Q14 y Q15).

4.1 Consultas analíticas

Q1 — Pacientes activos junto con sus propietarios

Resuelta con un traversal en Neo4j sobre la relación (:Propietario)-[:DUEÑO_DE]->(:Paciente {activo: true}) y enriquecida en MongoDB con raza y fecha_nac. Devuelve los 16 pares propietario-paciente activos del padrón. Se muestran los primeros registros:

Propietario	Ciudad	Paciente	Especie	Raza
Ana Rodríguez	Buenos Aires	Firulais	Perro	Labrador
Roberto Díaz	Rosario	Michi	Gato	Siamés
Roberto Díaz	Rosario	Pipo	Perro	Poodle
Valentina Molina	Córdoba	Nube	Conejo	Angora
Diego Herrera	Mendoza	Rex	Perro	Pastor Alemán
Paula Sosa	La Plata	Luna	Gato	Persa

Q2 — Consultas en estado *Seguimiento

Pipeline en MongoDB con \$match sobre estado: "Seguimiento" y dos \$lookup para traer veterinario y paciente.

Fecha	Paciente	Veterinario	Especialidad	Diagnóstico	Costo
2026-05-18	Simba	Cecilia Ríos	Dermatología	Dermatitis alérgica	\$4.800
2026-04-20	Rocky	Gabriela Fontana	Clínica General	En recuperación	\$4.200
2026-04-12	Mia	Cecilia Ríos	Dermatología	Dermatitis atópica	\$5.800
2025-11-10	Rex	Lorena Ibáñez	Clínica General	Gastritis	\$5.500
2025-11-01	Nube	Gabriela Fontana	Clínica General	Estrés ambiental	\$3.800

Q3 — Historial clínico unificado de *Firulais* (paciente 1)

Aplica \$unionWith sobre las colecciones consultas, vacunaciones y cirugías con un \$project que normaliza los campos al mismo esquema. Devuelve la línea de tiempo completa del animal:

Fecha	Tipo	Detalle	Resultado	Costo	Veterinario
2026-04-10	Cirugía	Mastectomía	Completada	\$52.000	Patricia Núñez
2025-12-02	Consulta	Control post-vacuna	Sin novedades	\$2.000	Gabriela Fontana
2025-10-05	Consulta	Control anual	Sano	\$4.500	Gabriela Fontana
2025-10-05	Vacunación	Antirrábica	—	—	Gabriela Fontana
2025-10-05	Vacunación	Sextuple	—	—	Gabriela Fontana
2025-08-15	Cirugía	Esterilización	Completada	\$25.000	Hernán Castillo

Q4 — Propietarios con más de un paciente

Conteo de la relación DUEÑO_DE agrupado por propietario.

Propietario	Ciudad	Total pacientes
Roberto Díaz	Rosario	2
Ana Rodríguez	Buenos Aires	2

Q5 — Veterinarios con consultas en los últimos 60 días

Pipeline en Mongo con \$match por fecha y \$group para acumular el total de atenciones, completado con \$lookup a veterinarios.

Veterinario	Especialidad	Sucursal	Consultas
Gabriela Fontana	Clínica General	Palermo	2
Cecilia Ríos	Dermatología	Palermo	2
Isabella Cruz	Clínica General	Palermo	1
Hernán Castillo	Cirugía	Belgrano	1
Lorena Ibáñez	Clínica General	Caballito	1
Natalia Ruiz	Clínica General	Belgrano	1

Q6 — Pacientes con vacunas vencidas

Filtra vacunaciones cuya próxima dosis es anterior a la fecha de hoy. Pensada como alerta operativa: incluye el teléfono del propietario para contactarlo.

Paciente	Especie	Propietario	Teléfono	Vacuna	Vencida
Max	Perro	Lucía Fernández	1101007890	Antirrábica	2026-03-15
Max	Perro	Lucía Fernández	1101007890	Sextuple	2026-03-15
Mia	Gato	Martín García	1112008901	Triple Felina	2026-04-10
Rocky	Perro	Sofía Torres	1123009012	Antirrábica	2026-05-20

Q7 — Top 5 diagnósticos más frecuentes

\$group por diagnóstico y \$sort descendente.

Diagnóstico	Frecuencia
Sano	5
Dermatitis atópica	2
Mejoría notable	1

Cicatrización normal	1
Estrés ambiental	1

Q8 — Productos con stock crítico

Filtro directo sobre stock_farmaceutico exigiendo unidades < 50, ordenado ascendente para resaltar los casos más urgentes.

Producto	Categoría	Unidades	Proveedor	Vencimiento
Tramadol 100mg	Analgésico	15	MediAnimal	2026-04-30
Acepromazina 1%	Tranquilizante	18	VetFarma SA	2026-09-15
Ketamina 10%	Anestésico	25	MediAnimal	2026-05-01
Prednisolona 5mg	Antiinflamatorio	28	BioVet SRL	2026-07-20
Enrofloxacin 50mg	Antibiótico	35	VetFarma SA	2026-08-15
Frontline Spray	Antiparasitario	40	MediAnimal	2027-01-10
Fenobarbital 30mg	Antiepiléptico	45	MediAnimal	2026-11-20

Q9 — Consultas de control con costo menor a \$5.000

Filtra por motivo que arranca con Control y costo bajo el umbral.

Fecha	Paciente	Veterinario	Motivo	Costo
2025-12-02	Firulais	Gabriela Fontana	Control post-vacuna	\$2.000
2026-05-25	Rex	Natalia Ruiz	Control gastritis	\$2.500
2026-05-02	Princesa	Isabella Cruz	Control rutina	\$2.800
2026-05-22	Thor	Gabriela Fontana	Control peso	\$3.000
2026-05-15	Lola	Lorena Ibáñez	Control anual	\$3.200
2026-04-05	Max	Isabella Cruz	Control anual	\$3.500
2026-04-20	Rocky	Gabriela Fontana	Control post-operatorio	\$4.200
2025-10-05	Firulais	Gabriela Fontana	Control anual	\$4.500

Q10 — Pacientes atendidos por sucursal (*Palermo*)

Recorrido en Neo4j que enlaza la sucursal con sus veterinarios y, a través de ellos, con las consultas y los pacientes que pasaron por allí.

Paciente	Especie	Atendido por
Firulais	Perro	Gabriela Fontana
Luna	Gato	Cecilia Ríos
Max	Perro	Isabella Cruz
Mia	Gato	Cecilia Ríos
Michi	Gato	Cecilia Ríos
Nube	Conejo	Gabriela Fontana
Princesa	Gato	Isabella Cruz
Rocky	Perro	Gabriela Fontana
Simba	Gato	Cecilia Ríos
Thor	Perro	Gabriela Fontana

Q11 — Ingresos por veterinario en el mes actual

Combina consultas y cirugías con \$unionWith, filtra los registros del mes corriente (junio de 2026 en la corrida) y agrupa por veterinario para reportar el total facturado y la cantidad de actos médicos. Las cirugías concentran el grueso de la facturación por su mayor ticket promedio.

Veterinario	Especialidad	Sucursal	Actos	Ingresos
Hernán Castillo	Cirugía	Belgrano	1	\$35.000
Patricia Núñez	Cirugía	Belgrano	1	\$28.000
Gabriela Fontana	Clínica General	Palermo	2	\$8.300
Cecilia Ríos	Dermatología	Palermo	1	\$5.200
Isabella Cruz	Clínica General	Palermo	1	\$3.500
Natalia Ruiz	Clínica General	Belgrano	1	\$2.800

Q12 — Propietarios sin consultas en el último año

Resuelto en Neo4j con un patrón WHERE NOT EXISTS que descarta a quien tenga al menos una consulta posterior al umbral de 365 días.

Propietario	Ciudad
Tomás Juárez	Salta
Claudia Reyes	Tucumán
Ricardo Silva	Rosario

4.2 Servicios operativos

Las funcionalidades Q13, Q14 y Q15 son servicios de escritura y, por lo tanto, se validaron ejecutando casos de uso representativos sobre ambos motores.

Q13 — ABM de propietarios (MongoDB + Neo4j)

Se ejecutó la secuencia completa: alta de un propietario nuevo con un paciente asociado, modificación de sus datos y baja lógica.

Operación	Resultado	Observación
Alta de Mariana Vega con su paciente Bruno	success — propietario_id 17, pacientes_registrados \[19\]	Se generan documento e ID en MongoDB y nodos :Propietario y :Paciente con la relación :DUEÑO_DE en Neo4j.
Modificación del teléfono y ciudad del propietario 17	success — actualizado en ambas bases	El servicio filtra qué propiedades corresponden al nodo del grafo y aplica \$set solo sobre esas.
Baja lógica del propietario 17	error — <i>"No se puede dar de baja al propietario. Tiene 1 paciente(s) activo(s) a su cargo."</i>	La validación previa en Neo4j impide dejar pacientes huérfanos y demuestra que la regla de negocio se aplica antes de tocar datos.

Q14 — Alta de consulta médica

El servicio recibe los datos de una atención (id_paciente, id_vet, fecha, motivo, diagnóstico, costo y estado) y, antes de escribir, valida en Neo4j que tanto el paciente como el veterinario existan y estén activos. Si la validación pasa, inserta el documento en la colección consultas de MongoDB y crea el nodo :Consulta junto con las relaciones :ATENDIDO_EN y :REALIZADA_POR en Neo4j. Se ejecutaron tres casos representativos para cubrir tanto el camino exitoso como los caminos de error:

Caso	Entrada	Resultado
------	---------	-----------

Paciente y veterinario válidos	id_paciente=1, id_vet=1, fecha 2026-06-08, motivo "Control anual", costo \$4.500	success — id_assigned 24, <i>"Consulta 24 registrada exitosamente para el paciente 1."</i>
Paciente inexistente	id_paciente=999, id_vet=1	error — <i>"El paciente con ID 999 no existe o no se encuentra activo."</i>
Veterinario inexistente	id_paciente=1, id_vet=999	error — <i>"El veterinario con ID 999 no existe o no se encuentra activo."</i>

Q15 — Decremento de stock farmacéutico

Operación atómica sobre stock_farmaceutico con \$inc negativo, condicionada a que el producto exista y tenga stock suficiente.

Caso	Resultado
Descontar 3 unidades del producto 1	success — <i>"Se descontaron 3 unidades del producto 1 con éxito."</i>
Stock insuficiente	error con detalle (stock_disponible, cantidad_solicitada)
Producto inexistente	error con mensaje <i>"El producto con ID X no existe en la base de datos."</i>

4.3 Observaciones sobre la consistencia cross-engine

Los tres servicios de la sección 4.2 escriben en ambos motores dentro del mismo flujo. Para garantizar que un fallo parcial no deje el sistema en estado inconsistente se aplicó un patrón de saga con compensación: si la escritura en Neo4j falla luego de haber persistido en MongoDB, el servicio revierte la operación en Mongo (delete_one para altas y replace_one con el snapshot previo para modificaciones y bajas). El caso de la baja del propietario 17, devuelto correctamente como error por la validación previa, confirma además que las reglas de negocio se evalúan antes de iniciar las escrituras. Esta combinación de validación temprana y compensación posterior fue suficiente para que el backoffice opere con coherencia eventual robusta sin necesidad de un coordinador transaccional distribuido.

5. Conclusiones

La realización de este proyecto permitió consolidar de forma práctica la teoría del diseño de bases de datos NoSQL mediante la construcción de un sistema bajo el esquema de persistencia políglota.

La combinación estratégica de MongoDB y Neo4j demostró ser una solución óptima para los requerimientos de VetSalud S.A.. MongoDB absorbió con naturalidad la flexibilidad requerida para el catálogo y control de stock de productos farmacéuticos y los esquemas

maestros de las entidades independientes. Al mismo tiempo, Neo4j mitigó el costo de cómputo derivado de las relaciones complejas e históricas de la clínica veterinaria, permitiendo consultas inmediatas sobre líneas de tiempo de pacientes y análisis relacionales complejos. El desarrollo bajo este enfoque simuló con éxito un escenario corporativo real y escalable.